

Fulton Hogan Roadway Construction Analysis

Technical Framework

Authors: Brayden Davies (74597743, bda79), David Ewing (82171165, dew59), Ann Benji (67936363, abe167), Matthew Madriaga (11720432, jma538)

Date: 11th October 2024

Executive Summary

The report presents an analysis conducted for Fulton Hogan to determine the most suitable time for scheduling roadworks in the Central Business District (CBD) of Auckland, Wellington and Christchurch, using telecommunications data from spark and Vodafone. The goal of this project is to optimize roadwork scheduling by minimizing traffic disruption, reducing costs, and ensuring safety for both workers and road users.

Data from multiple sources, including Telecommunications data from spark and Vodafone, Statistical Area and Territorial Authority Data, Populations estimates and urban rural indicators was utilised. The data pre processing phase involved cleaning and preparing these data sets to ensure accuracy and consistency. Challenges such as missing or duplicate data were resolved through validation. Functions were developed to standardise data, flag, and remove errors and merging data sets. We generated maps for each CBD to identify the relevant regions, after which the analytical process could begin.

Introduction

Fulton Hogan undertakes significant roadworks across major cities in New Zealand, including Auckland, Wellington and Christchurch. These roadworks can be highly disruptive due to a variety of factors, such as costs, traffic management, and safety risks. This investigation focuses on finding the most appropriate time to plan roadworks in these cities' CBDs.

Roads in these busy urban centres often deteriorate over time or require upgrades to accomodate increasing traffic or new development projects. Regular maintenance is essential to ensure safe driving conditions, improve traffic flow, and adapt to new infrastructure requirements. The objective of this investigation is to identify optimal roadwork schedules that minimises costs, improve worker safety, and reduce traffic disruption.

We achieve this by leveraging telecommunications data from Spark and Vodafone, along with Statistical (SA2) and Territorial (TA) Area data, to determine the number of mobile phones present within specific SA2 regions.

Data Description

The data sets used in the analysis include:

- **sp_data.csv.gz** - Telecommunications data from Spark
- **vf_data.parquet** - Telecommunications data from Vodafone
- **sa2_2023.csv** - This file contains the Statistical Area Level 2 (SA2) codes with their corresponding regions recorded in 2023.
- **sa2_ta_concord_2023.csv** - This concordance report contains the SA2 codes with their corresponding regions. Additionally, it has the Territorial Authority (TA) code corresponding to its territory name. We also get the relationship of the SA2 codes to the TA codes on the map.
- **subnational_pop_ests.csv** - The subnational population estimates contain population information like gender, age and population information for the corresponding SA2 regions.

- **urban_rural_to_indicator_2023.csv** - This CSV file gives us an insight into the various types of settlement across the statistical regions of New Zealand.
 - **urban_rural_to_sa2_concord_2023.csv** - This concordance report gives the relationship between the urban rural code and the SA2 code for all the regions in New Zealand.
-

Data Preprocessing

This section outlines the data preparation process, which included cleaning and validating the datasets. Challenges such as missing values, deuplicate records, and incorrect column names were addressed using a series of scripts developed for standardising the data.

Refer to Ewing, D. (2024). Narrative and debugging overview of cellphone data cleaning functions for a detailed description of the data cleaning process.

(20241010_Narrative_and_Debugging_Overview_of_Cellphone_Data_Cleaning_Functions.pdf)

Initialisation Phase (00__initialise.R)

An initialisation script that sets up essential configurations and imports the libraries for the entire pipeline. The script sets up environment configurations, such as establishing paths, defining global variables, and importing libraries like dplyr, ggplot2, and vroom that are used throughout the process.

Importing Cell Data (001__import__celldata.R)

The import phase brings raw data into the R environment from sources like CSV files, RDS files, or Parquet files using utilities like read_csv(), read_rds(), or arrow::read_parquet() to import the data. It also includes data checks such as validating file existence and confirming that data is correctly formatted, with the expected number of columns and column names.

Overview of Functions in 002__clean__celldata.R

The functions in 002__clean__celldata.R are designed to clean and preprocess data from two vendors' cellphone datasets, each containing columns representing time, SA2 codes, and the number of calls on an hourly basis for two weeks. Given the numerous issues with NA and zero values, as well as multiple duplicate entries in the data, these functions are structured to handle cleaning, validation, and filtering in a stepwise manner. Below is a detailed description of each function along with their interactions as they are called in the overarching script run.R.

The following files performed are the subnational_pop_ests and sa2_2023 files.

Read Phase (000__read__files__BD.R)

The functions read_sa2_2023 and read_subnational_pop create the necessary CSV files. The first input file sa2_2023.csv is read and dynamically named to the current date to ensure version control. It checks the existence the input file to prevent processing if the file is missing. Also, it removes any pre-existing RDS file with the same name. The SA2 data is further read into a tibble for easy manipulation and saved to an RDS file which is optimal for further processing.

Similarly, the second function reads subnational_pop_ests.csv has the same processes applied, with dynamic naming and output RDS file, checks if the file exists and avoids overwriting existing files. This data is eventually read into a tibble and saved as an RDS file.

Clean Phase (002__clean__files__BD.R)

The pipeline below saves the cleaned data to new CSV files.

The cleaning phase involves checking for any data quality issues present in the two files. This includes identifying invalid and incorrect names used to define the columns, missing values that are marked as NA,

and duplicate data that appears repetitively. This initial step is to read the files so that the script knows which data files have to be cleaned.

The first input file, `sa2_2023.csv`, dynamically generates to ensure flexibility. New columns are defined above to give meaningful identifiers, which improves both readability and the overall structure of the dataset. The first 6 rows are skipped to read only the relevant fields. The cleaned dataset is saved as `clean_sa2_2023_data.csv` which can be used for further analysis.

The second input file, `subnational_pop_estimates.csv`, is also dynamically generated to provide flexibility in handling population estimates data. The new column names are defined to improve interpretability and also filter out any incomplete or irrelevant data. Additionally, the numeric codes are standardized to ensure consistency across the other datasets. This final cleaned dataset is saved as `clean_subnational_pop_data.csv` for subsequent analysis.

This pipeline ensures that the above datasets are well – organized, with meaningful columns and only contain relevant information which is ready for further analysis.

The following files performed is `urban_rural_to_sa2_concord_2023`.

Read Phase (000_initialise_AB.R)

Readr and tidyverse are installed to perform data reading and writing operations. It captures the current system data to create a unique, time- stamped filename when saving the processed data. Then it reads the input CSV file to keep it in memory in case it is needed for further processing or analysis and written to a new file which omits row names to improve compatibility with other files. However, since most of the information was not required, further analysis of the file was halted.

Read_phase(000_read_CONCORD_AB.R)

This pipeline is read a CSV file “`urban_rural_to_sa2_concord_2023.csv`”, along with a boolean to save the resulting data as a RDS file. The input and output file paths are constructed dynamically with the folder and filenames, which includes the date, ensuring that the output file is unique to the day it is created, helping with version control and avoids files not to be overwritten. It further checks that no operations are performed on missing files and removes existing RDS file of the same name, ensuring that outdated or incorrect files are not in the directory. Eventually, it extracts the column specification, which allows the user to inspect or work with the column types later during processing. The resulting dataset is saved as an RDS file, which is efficient for storing R data objects and often preferred during processing.

The following files performed are the `sa2_ta_concord_2023` and `urban_rural_to_indicator_2023` files.

Read_phase(000_initialise_MM.R)

The pipeline begins by conditionally loading packages only if it is required, which makes the script portable and reproducible on different systems. This pipeline processes 2 specific CSV files which are processed separately.

The first input file `sa2_ta_concord_2023.csv` and output file is dynamically named to ensure version control. Following which it checks if the file exists or halts if the file is missing. To prevent overwriting, it removed the RDS file with the same name. The concordance data is read into a tibble and the RDS file is compressed for optimized for loading in following workflows. The final processed data returns a tibble for immediate use.

The second input file `urban_rural_to_indicator_2023.csv` has defined output and RDS files like before along with including the current date for clear version control. The pipeline checks in case of missing files to avoid further errors. This also removes pre – existing RDS files with the same name to avoid confusion from older versions. Following, this is read into memory as a tibble in order to be easily manipulated further and saved as an RDS file for long term storage and returns for downstream analysis.

Clean_phase (002_clean_MM.R)

The pipeline below is designed to save the cleaned data to new CSV files.

The first input file, `sa2_ta_concord_2023.csv` is dynamically generated to ensure flexibility. We define new columns to provide clear and meaningful identifiers for each column, thus improving readability and maintaining the data. Further, we skipped the first 7 rows to apply the new column names and saved to the new CSV file `clean_sa2_ta_concord_data.csv`.

The second input file `urban_rural_to_indicator_2023.csv` is dynamically generated to generate flexibility while handling further analysis. We define new column names for each data field, making sure that the dataset is easily interpretable, and the information is well – organised. Again, the first seven rows are skipped, and the new column names are applied, removing an unnecessary column to keep only relevant columns for analysis. Finally, the cleaned dataset is saved to the new CSV file `clean_urban_rr_indicator_data.csv`

Building of the Map

To identify the CBD regions of Auckland, Wellington and Christchurch, the process was completed in three steps:

1. Creating a polygon of the CBD:

To get a clear picture of which regions exactly are covered to give the polygon of each CBD, we have found the boundary region coordinates of each CBD, which gave CBD polygon on the map of New Zealand.

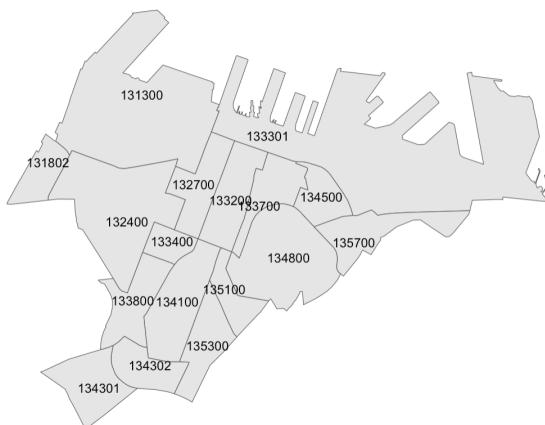
2. Finding the SA2 Codes for the CBD:

The polygon found in the previous step created a boundary that could be used to find the corresponding SA2 codes. To help us graph the SA2 codes, we used a map of New Zealand comprised of polygons that surround the Statistical Areas (SA2). This map was found on StatNZ, and by clicking on the polygons that are within the polygon found earlier, we found the SA2 codes.

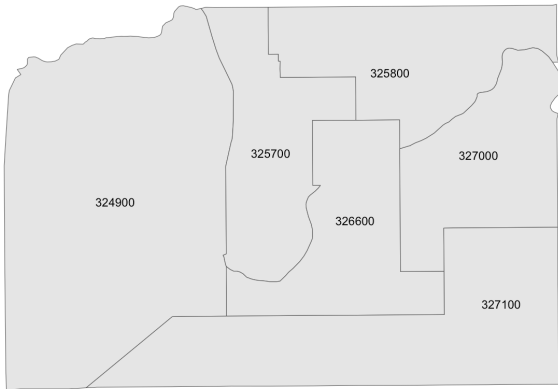
3. Creating a dataframe for the CBD:

After finding the correct SA2 codes for each CBD, we create a dataframe for Auckland, Wellington and Christchurch CBD by filtering out data that wasn't within the CBD boundaries.

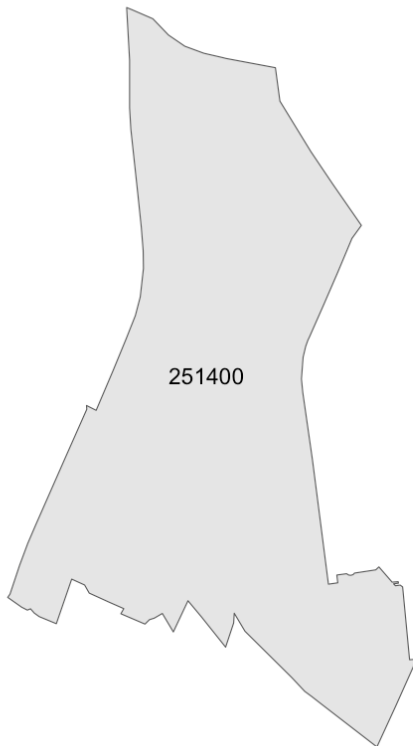
Map of Auckland CBD



Map of Christchurch CBD



Map of Wellington Central (251400)



Main RScripts

The purpose of these scripts is so that it creates a suitable environment that contains everything necessary for the project to fully function. These functions can all be ran by one rscript, 'run.R'. These rscripts get ran sequentially. Each rscript requires some data from it's predecessor, so it's important that the previous rscript is ran before running anything else. For any sourced rscript, functions can be called from them so long as the function name is used when being called.

00_initialise: Loads in required packages. By loading the required packages, we set up the environment for our project to run more smoothly. The packages installed are; dplyr, tools, arrow, broom, readr, lubridate. If these aren't already in the environment they are installed. This file also offers conflict management for functions

01_import: By using the initialise files mentioned earlier, we can read the required datasets for our project. This rscript calls all the rscripts who read files, e.g 000_initialise_MM.R, and executes their functions in this single rscript, retrieving the required data. This includes sa2_ta_concord_2023, urban_rural_to_indicator, subnational_pop_est, vf.parquet and sp_data.csv.gz. Importing the datasets allows for further analysis. We use source() to import functions and code from other scripts

02_clean: This script is responsible for cleaning the data, removing any na values, duplicates and extra columns. By using e.g 002_clean_files_MM.R script it cleans the datasets that were previously imported which reduces chances of errors and improves analysis.

03_analyse: This is used to produce datasets related each required CBD. For example, this file retrieves all the SA2 Codes relating to the Auckland CBD, creating a dataset for it. A shapefile is also read, which requires the 'sf' package, to be able to make maps. Cell data also gets analysed to help for visualisations.

04_visualise: This script uses each CBD dataframe created previously. A leaflet map is used here to create an interactive map using a dataset that contains coordinates found online. The shapefile data implemented from the previous 03_analys script to make ggplot maps for each CBD, with each area in the map containing their respective SA2 Codes. Example of function sourced: 000_wellington_cbd.R.

Conclusion

This report provides an analysis aimed to optimize roadwork scheduling in the CBDs of Auckland, Wellington, and Christchurch. Throughout the project, we had to process the data, tackling challenges such as missing values, duplicates and inconsistencies in column names to ensure data quality. By applying a cleaning method and validation, we created a foundation for our analysis.

The process of building a map of the CBD regions allowed us to visualize the areas impacted by roadworks and align them with telecommunications data. The geographical representation will assist Fulton Hogan in making informed decisions. Moving forward, it is recommended that Fulton Hogan continue to utilize this framework in future roadworks project.

References

1. Ewing, D. (2024). Narrative and debugging overview of cellphone data cleaning functions.